

EV Routing System with SG-GAN & GNN

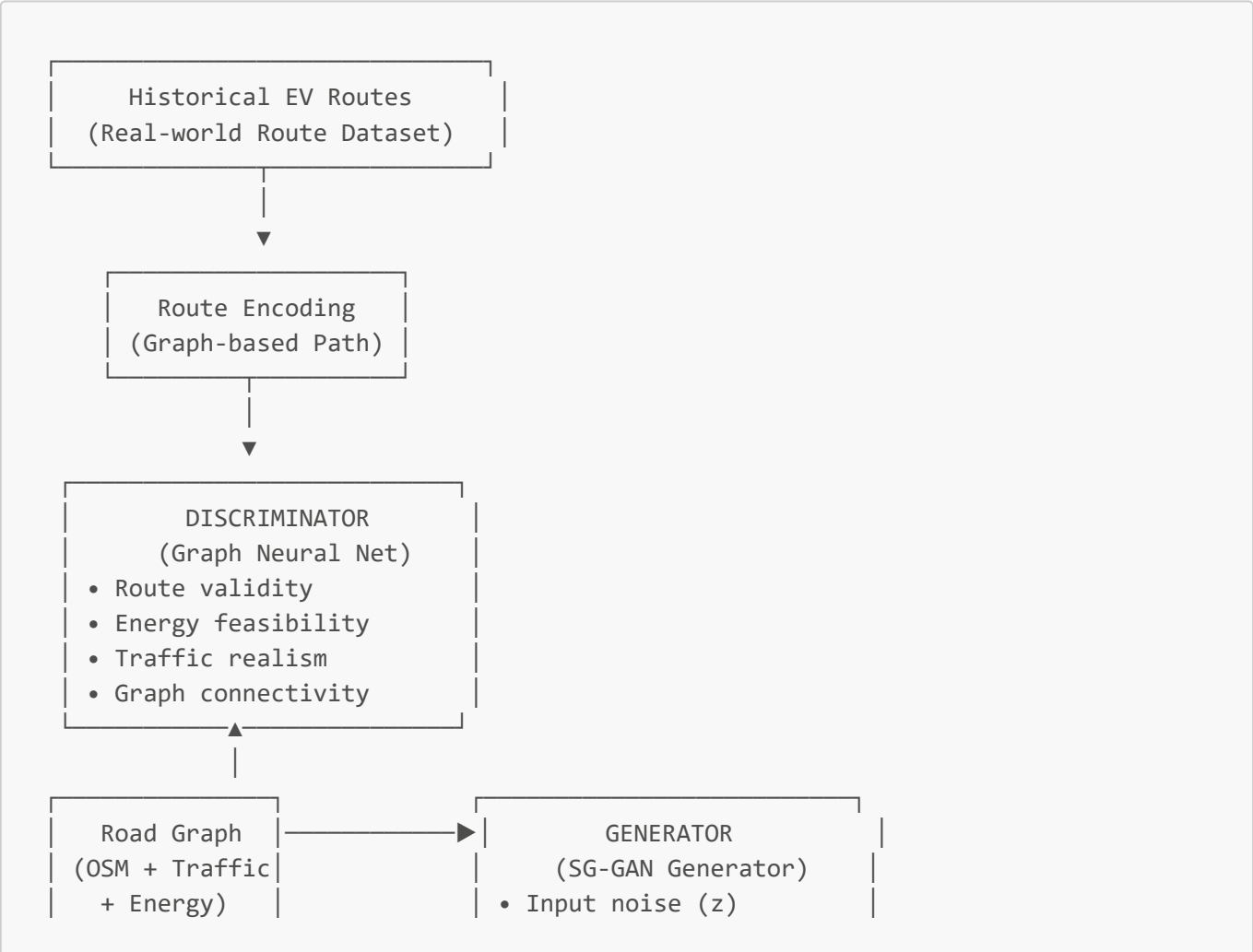
AI-Powered Electric Vehicle Route Optimization using Graph Neural Networks and Semi-supervised Generative Adversarial Networks

 Table of Contents

- 1. [Overview](#)
- 2. [Architecture](#)
- 3. [Installation](#)
- 4. [Quick Start](#)
- 5. [Detailed Usage](#)
- 6. [Project Structure](#)
- 7. [Components](#)
- 8. [How It Works](#)
- 9. [Future Updates](#)
- 10. [Troubleshooting](#)

 Overview

This system implements the complete architecture from the diagram:



- Source & Destination
- EV battery state
- Traffic conditions

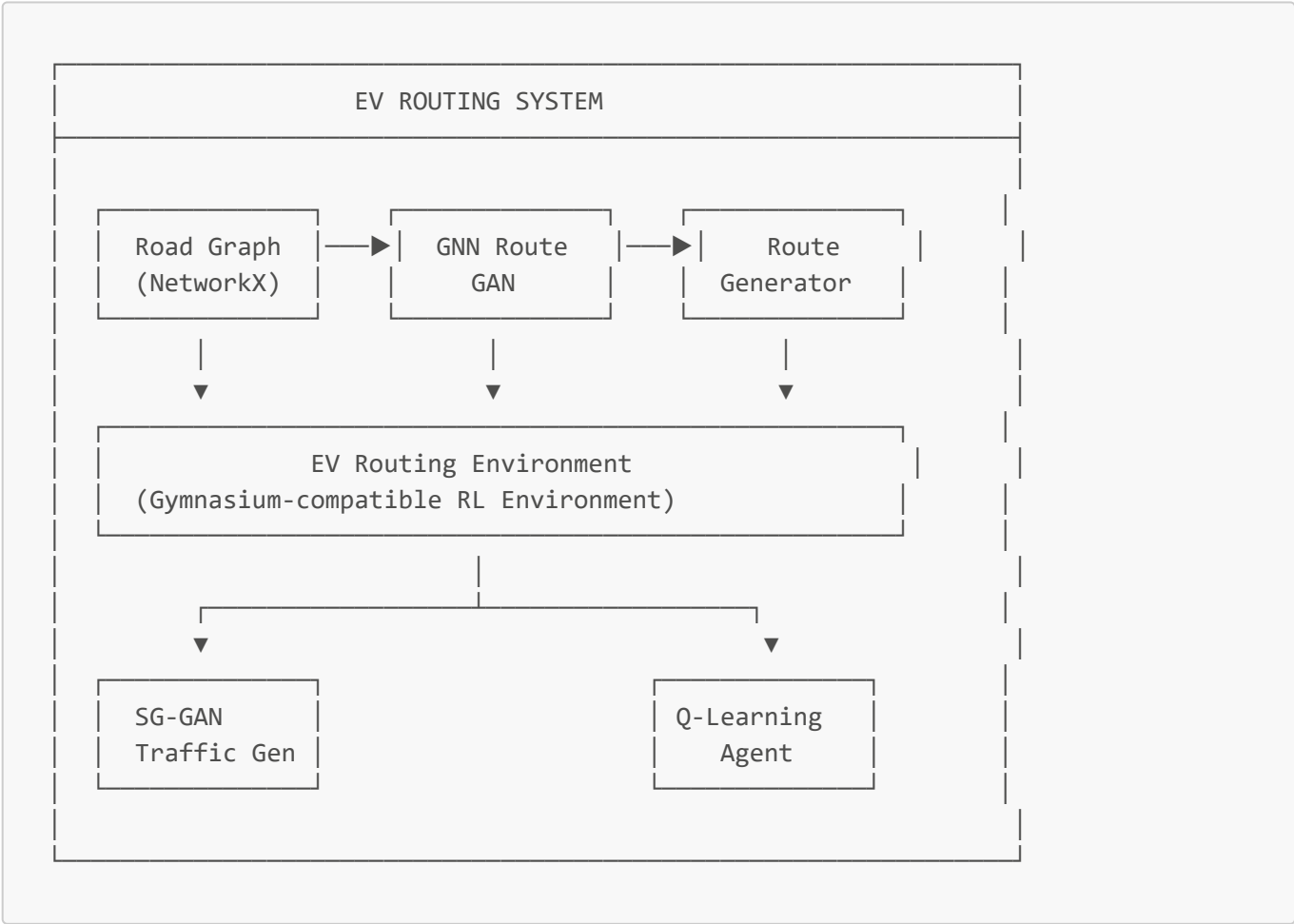
→ Generates EV routes

Key Features

Feature	Description
GNN Route GAN	Graph Neural Network GAN for direct route generation
SG-GAN Traffic	Traffic pattern generation with temporal variations
Graph-Based Network	NetworkX road topology with energy/time attributes
Q-Learning Agent	Reinforcement learning for route optimization
Multi-Criteria Routing	Shortest, fastest, energy-optimal routes
Charging Integration	Smart charging stop planning

 Architecture

System Components



GNN Route GAN Architecture

GENERATOR (GNNRouteGenerator)

Input:

- Noise vector $z \sim N(0,1)$
- EV State (battery SoC, energy, range, time, rate)
- Source/Destination encoding

Processing:

- Input Dense layers
- Graph Convolutional Layers (GCN)
- Graph Attention Layer (GAT)

Output:

- Node probability distribution
- Route sequence via greedy decoding

DISCRIMINATOR (GNNRouteDiscriminator)

Input:

- Route node features
- Adjacency matrix
- EV State

Processing:

- Node encoder
- GCN layers
- GAT layer
- Global pooling

Outputs (Multi-task):

- Route validity score
- Energy feasibility score
- Traffic realism score
- Graph connectivity score

Installation

Prerequisites

- **Python 3.9+** (recommended: 3.10 or 3.11)
- **pip** package manager
- **8GB+ RAM** recommended
- **Optional:** NVIDIA GPU with CUDA

Step-by-Step Installation

```
# Step 1: Navigate to project directory
cd c:\Users\thavv\OneDrive\Desktop\project_1\EV_Routing

# Step 2: Create virtual environment
python -m venv .venv

# Step 3: Activate virtual environment
# Windows:
.venv\Scripts\activate
# Linux/Mac:
# source .venv/bin/activate

# Step 4: Install dependencies
pip install -r requirements.txt

# Step 5: Verify installation
python -c "import tensorflow as tf; import networkx as nx; import gymnasium as gym; print('✅ All dependencies installed!)"
```

Verify TensorFlow GPU (Optional)

```
python -c "import tensorflow as tf; print('GPU Available:',
tf.config.list_physical_devices('GPU'))"
```

 Quick Start

Train the Complete System

```
cd src
python main.py --mode train
```

This executes the **7-step training pipeline**:

Step	Description	Output
1	Create Road Network	results/plots/road_network.png
2	Generate Traffic Data	data/training_data/traffic_data.npy
3	Train SG-GAN	models/sg_gan/traffic_gan_*.keras
3b	Train GNN Route GAN	models/gnn_gan/*.weights.h5
4	Create RL Environment	Environment ready
5	Train Q-Learning Agent	models/q_learning/trained_agent.pkl

Step	Description	Output
6	Create Route Generator	Route planner ready
7	Evaluate System	results/metrics/*.json

Quick Training (For Testing)

```
python main.py --mode train --grid-size 5 --gan-epochs 5 --episodes 50
```

Run Demo

```
python main.py --mode demo
```

Run Tests

```
python main.py --mode test
```

Evaluate Models

```
python main.py --mode evaluate
```

 Detailed Usage

Command Line Options

```
python main.py --help
```

Option	Description	Default
--mode	train, evaluate, demo, test	train
--grid-size	Road network grid size (NxN)	10
--gan-epochs	SG-GAN training epochs	50
--episodes	RL training episodes	500

Training Configurations

```
# Small (quick testing)
python main.py --mode train --grid-size 5 --gan-epochs 10 --episodes 100

# Medium (balanced)
python main.py --mode train --grid-size 10 --gan-epochs 50 --episodes 500

# Large (thorough training)
python main.py --mode train --grid-size 15 --gan-epochs 100 --episodes 1000
```

Using Individual Components

1. Road Graph

```
from road_graph import RoadGraph, EVState

# Create network
graph = RoadGraph(grid_size=10)
print(f"Nodes: {graph.num_nodes}, Edges: {graph.num_edges}")

# Find shortest path
path = graph.shortest_path(0, 99)

# Create EV state
ev = EVState(battery_soc=80, current_node=0)
```

2. GNN Route GAN

```
from gnn_route_generator import GNNRouteGAN, generate_historical_routes
from road_graph import RoadGraph
import numpy as np

# Create road network
graph = RoadGraph(grid_size=10)

# Generate historical routes for training
routes, ev_states = generate_historical_routes(graph, n_routes=500)

# Create and train GAN
gan = GNNRouteGAN(num_nodes=graph.num_nodes)
gan.train(graph, routes, ev_states, epochs=50)

# Generate new route
ev_state = np.array([0.8, 0.7, 0.6, 0.5, 0.15])
route = gan.generate_route(source=0, destination=99, road_graph=graph,
ev_state=ev_state)
```

3. SG-GAN Traffic

```
from traffic_generator import SGGANTrafficGenerator, create_synthetic_traffic

# Generate training data
traffic_data = create_synthetic_traffic(n_samples=1000)

# Train GAN
gan = SGGANTrafficGenerator()
gan.train(traffic_data, epochs=50)

# Generate traffic scenarios
generated = gan.generate_traffic_scenarios(n_samples=10)
```

4. Q-Learning Agent

```
from q_learning_agent import QLearningAgent, train_q_learning_agent
from environment import LegacyEVRoutingEnvironment

# Create environment
env = LegacyEVRoutingEnvironment(grid_size=5)

# Create and train agent
agent = QLearningAgent(action_space=5)
history = train_q_learning_agent(env, agent, episodes=500)

# Save model
agent.save_model('models/q_learning/my_agent.pkl')
```

Project Structure

```
EV_Routing/
├── data/
│   ├── generated_traffic/           # GAN-generated traffic
│   │   └── sample_traffic.npy
│   ├── historical_routes/          # Historical routes for GNN
│   │   └── routes.npz
│   └── training_data/              # Training data
│       └── traffic_data.npy
├── models/
│   ├── gnn_gan/                   # GNN Route GAN models
│   │   ├── gnn_generator.weights.h5
│   │   └── gnn_discriminator.weights.h5
│   └── q_learning/                 # Q-Learning agent
│       └── trained_agent.pkl
```

```
├── sg_gan/                                # SG-GAN models
│   ├── traffic_gan_generator.keras
│   └── traffic_gan_discriminator.keras
├── results/
│   ├── metrics/                          # Evaluation metrics
│   │   ├── evaluation_results.json
│   │   └── training_metrics.npz
│   └── plots/                            # Visualizations
│       ├── road_network.png
│       ├── training_progress.png
│       ├── gan_training.png
│       └── demo_routes.png
├── src/
│   ├── road_graph.py                    # Road network graph
│   ├── gnn_route_generator.py          # GNN Route GAN (NEW)
│   ├── traffic_generator.py            # SG-GAN traffic
│   ├── environment.py                  # RL environment
│   ├── q_learning_agent.py             # Q-Learning/DQN agents
│   ├── route_generator.py              # Route algorithms
│   ├── main.py                         # Main entry point
│   └── evaluate.py                     # Evaluation utilities
├── requirements.txt
└── README.md
```

Components

1. Road Graph ([road_graph.py](#))

Class	Purpose
RoadGraph	NetworkX road network with nodes, edges, charging stations
EVState	EV state dataclass (battery, position, time)
ChargingStation	Station with capacity and pricing
HistoricalRouteGenerator	Synthetic historical routes

2. GNN Route GAN ([gnn_route_generator.py](#)) - **NEW**

Class	Purpose
RouteEncoder	Encodes routes as graph data
GraphConvLayer	Graph convolutional layer
GraphAttentionLayerV2	Multi-head graph attention
GNNRouteGenerator	GNN-based route generator

Class	Purpose
GNNRouteDiscriminator	Multi-task route validator
GNNRouteGAN	Complete GAN for route generation

3. Traffic Generator ([traffic_generator.py](#))

Class	Purpose
SGGANGenerator	Traffic pattern generator
SGGANDiscriminator	Multi-task traffic validator
SGGANTrafficGenerator	Complete traffic GAN

4. Environment ([environment.py](#))

Class	Purpose
EVRoutingEnvironment	Gymnasium RL environment
LegacyEVRoutingEnvironment	Backward-compatible wrapper
EnvironmentConfig	Configuration dataclass

5. Q-Learning Agent ([q_learning_agent.py](#))

Class	Purpose
QLearningAgent	Tabular Q-Learning
DQNAgent	Deep Q-Network

6. Route Generator ([route_generator.py](#))

Class	Purpose
RouteCandidate	Single route with metrics
RouteGenerator	Multiple routing algorithms
EVRoutePlanner	High-level route planning

 How It Works

Training Pipeline

1. CREATE ROAD NETWORK

└─ Generate grid graph with edges, charging stations

2. GENERATE TRAFFIC DATA

- └ Synthetic traffic with rush hours, patterns
- 3. TRAIN SG-GAN
 - └ Learn traffic distribution
 - └ Generator: noise → traffic patterns
 - └ Discriminator: real vs fake traffic
- 3b. TRAIN GNN ROUTE GAN
 - └ Generate historical routes
 - └ Encode routes as graph data
 - └ Generator: conditions → route probabilities
 - └ Discriminator: validity, energy, realism, connectivity
- 4. CREATE RL ENVIRONMENT
 - └ State: EV position, battery, destination
 - └ Actions: move directions
 - └ Rewards: destination bonus, energy penalty
- 5. TRAIN Q-LEARNING
 - └ Explore environment
 - └ Learn optimal policy
 - └ Build Q-table
- 6. CREATE ROUTE GENERATOR
 - └ Combine: shortest path + GAN-guided + k-paths
- 7. EVALUATE
 - └ Test routes
 - └ Measure success rate
 - └ Save metrics

Route Generation Process

Input: Source, Destination, EV State

1. Classical Methods:
 - Shortest path (Dijkstra)
 - Energy-optimal path
 - Time-optimal path
 - K-shortest paths
2. GAN-Guided:
 - GNN Route GAN generates probability
 - Greedy decode with heuristics
 - Validate with discriminator
3. Charging-Aware:
 - If battery < 50%
 - Route via charging stations
4. Rank & Return:

- Score by energy, time, feasibility
- Return top N candidates



Future Updates

What to Update

Component	When to Update	How
Road Graph	Real data available	Add OSM integration
GNN Route GAN	More routes	Increase training data
SG-GAN Traffic	Real traffic	Integrate API data
RL Agent	Environment changes	Retrain

Priority Improvements

1. Real Road Data (HIGH)

```
# Add to road_graph.py
import osmnx as ox

def load_osm_graph(self, location="New York, USA"):
    G = ox.graph_from_place(location, network_type='drive')
    self.graph = G
```

```
pip install osmnx
```

2. Real Traffic API (HIGH)

```
# Add to traffic_generator.py
def load_traffic_api(api_key, location):
    # TomTom or Google Maps Traffic API
    pass
```

3. Improved GNN (MEDIUM)

- Add edge features (road type, speed limit)
- Use transformer architecture
- Multi-hop attention

4. Advanced RL (MEDIUM)

- Double DQN
- Dueling DQN
- PPO for continuous actions

Update Checklist

When updating:

- ☐ Backup `models/` folder
- ☐ Update `requirements.txt`
- ☐ Run `python main.py --mode test`
- ☐ Retrain affected models
- ☐ Update this README

Troubleshooting

Common Issues

1. TensorFlow Error

```
Error: No module named 'tensorflow'
```

Solution:

```
pip install tensorflow>=2.10.0
```

2. Memory Error

Solution: Reduce grid size:

```
python main.py --mode train --grid-size 5
```

3. GNN GAN Not Learning

Symptoms: D Loss $\rightarrow$ 0, G Loss increasing

Solution:

- Balance discriminator/generator training
- Add gradient clipping
- Increase batch size

4. Route Generation Slow

Solution: Already fixed in latest version. The `_k_shortest_paths` now uses early stopping.

5. Agent Not Learning

Solution:

- Increase episodes: `--episodes 1000`
- Check reward function
- Verify environment reset

Debug Mode

```
# In main.py, add:  
import logging  
logging.basicConfig(level=logging.DEBUG)
```

Example Output

After training:

```
=====
PIPELINE COMPLETE
=====
Total time: 0:05:51

 Results:
  Road Network: 100 nodes, 410 edges
  Charging Stations: 8

  SG-GAN: ☒ Trained
  GNN Route GAN: ☒ Trained (242 historical routes)
  Q-Learning: 78% success rate

  Route Generation:
    Avg candidates: 5
    Avg energy: 3.4 kWh
    Feasibility: 92%

  Saved files:
    - models/sg_gan/traffic_gan_*.keras
    - models/gnn_gan/*.weights.h5
    - models/q_learning/trained_agent.pkl
    - results/plots/*.png
    - results/metrics/*.json
```

License

This project is for educational and research purposes.

Happy Routing! 🚗 ⚡